

AI Interviewer Chatbot

1. Purpose

This blueprint is designed for training, to build an AI-based interview chatbot end-to-end. It focuses on core engineering concepts: LLM orchestration, state management, structured outputs, persistence, and basic evaluation.

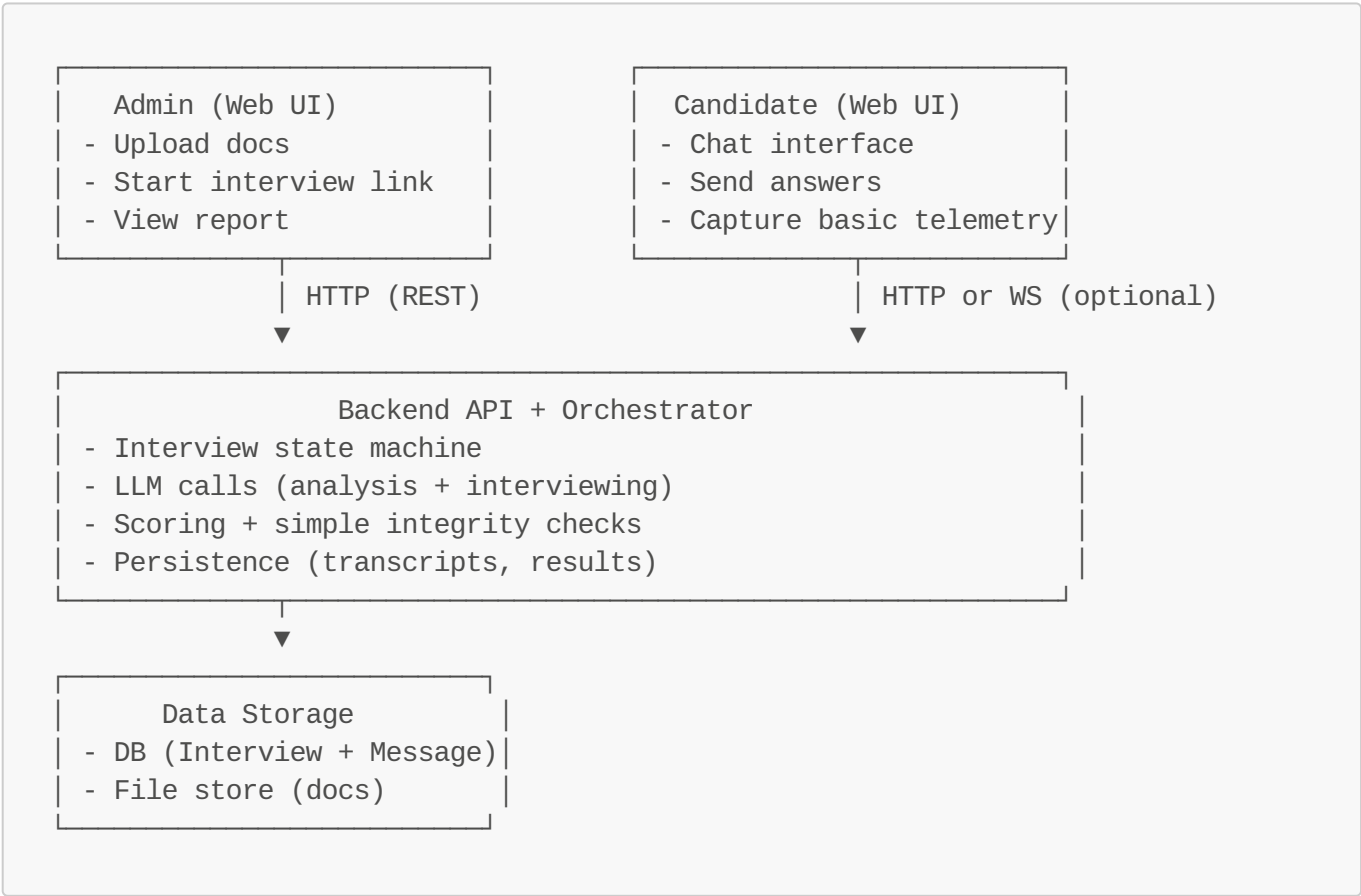
Learning Objectives

By completing this project, a developer should be able to:

- Design an app around an LLM with **deterministic workflow control**
- Use **structured schemas** (JSON) to make LLM outputs reliable
- Build an **adaptive interview loop** (difficulty + topic coverage)
- Persist transcripts and scoring in a database
- Add basic integrity signals (timing + paste detection)
- Generate a final report from transcript + rubric

2. Minimal System Overview

2.1 Simplified Architecture



2.2 Technology Suggestions (Non-prescriptive)

Pick a stack the developer can learn quickly:

- Frontend: React (or any web UI)
 - Backend: Python (FastAPI) or Node (Express/Nest)
 - DB: Postgres or SQLite (for training)
 - File storage: local filesystem (for training)
 - LLM: any one provider initially (add abstraction later)
-

3. Minimal Feature Set

Must-Have (Training MVP)

1. **Interview creation** (admin)
2. **Document ingestion** (upload resume + role description)
3. **Match analysis** (extract focus areas to guide interview)
4. **Interview chat loop** (ask Q → accept answer → score → next Q)
5. **Adaptive difficulty** (simple up/down rules)
6. **Transcript persistence** (messages + metadata)
7. **Final report generation**

Nice-to-Have (Stretch)

- WebSocket real-time chat
 - Additional integrity signals (focus/blur)
 - Multi-model routing
 - Rubric versioning
-

4. Core Workflows (Learning Version)

Workflow A — Create + Analyze

1. Admin creates an interview record.
2. Upload two documents:
 - Resume
 - Role description
3. Parse documents to text.
4. Call the LLM to generate:
 - role requirements summary
 - candidate summary
 - focus areas for interview (top 3–5)
5. Store match analysis on interview.

Workflow B — Interview Session

1. Candidate opens interview link (token optional for training).
2. Interview starts with an introduction and first question.
3. For each candidate answer:
 - persist message
 - score answer quality (1–10) using rubric
 - adjust difficulty
 - generate next question (target topic coverage)
4. Stop after N questions.

Workflow C — Report

1. Aggregate per-question scoring.
2. Summarize strengths, weaknesses, and recommendation.
3. Store and display report.

5. State Machine (Simple)

States

- **DRAFT**: created
- **READY**: documents analyzed
- **IN_PROGRESS**: candidate session active
- **COMPLETED**: interview finished, report ready

Transitions

- DRAFT → READY → IN_PROGRESS → COMPLETED

6. Training Data Model (Conceptual)

Entities

Interview

- id
- status
- resume_path
- role_path
- match_analysis_json
- target_questions (e.g., 8)
- difficulty_start (e.g., 5)
- report_json

Message

- id

- interview_id
- role (assistant | candidate)
- content
- timestamp
- question_number (nullable)
- difficulty_level (nullable)
- answer_quality_score (nullable)
- telemetry:
 - response_time_ms
 - paste_detected

Why This Model

- Small enough to implement quickly.
 - Still demonstrates the key patterns: state, transcript, scoring, metadata.
-

7. Rubric and Scoring (Learning-Friendly)

7.1 A Minimal Rubric

Use 3 competencies for training:

- **Technical correctness** (0–10)
- **Problem solving / reasoning** (0–10)
- **Communication clarity** (0–10)

Overall answer score can be average or weighted.

7.2 Structured Per-Answer Evaluation

Require the model to return **strict JSON**:

- score: 1–10
 - rationale: 2–4 sentences
 - evidence: short excerpt from answer
 - followup_hint: what to probe next
-

8. LLM Usage Patterns (Core Learning)

8.1 Structured Output Contracts

Use schemas to prevent messy outputs.

Example JSON contract (concept)

```
{
  "score": 7,
  "rationale": "...",
  "evidence": "...",
  "followup_hint": "..."
}
```

8.2 Three LLM Calls (Minimum)

1. **Document analysis:** summarize role + candidate, produce focus areas.
2. **Answer evaluation:** grade answer against rubric.
3. **Question generation:** produce next question given focus areas + difficulty.

8.3 Minimal Adaptive Difficulty Rule

```
if score >= 7 → difficulty += 0.5
if score <= 4 → difficulty -= 0.5
clamp difficulty between 3 and 10
```

9. Basic Integrity Signals (Training Version)

Keep it simple and educational:

- **response_time_ms:** time from first keystroke to send
- **paste_detected:** boolean on paste event

Use these only to add a small “integrity note” in the report:

- “Some answers were very fast relative to length”
- “Paste events detected”

No automatic disqualification.

10. UI Requirements (Minimal)

Admin UI

- Create interview
- Upload resume + role description
- View match analysis
- Open candidate link
- View final report

Candidate UI

- Chat window
 - Message input
 - Capture paste event + basic timing
-

11. Suggested Learning Roadmap (Hands-On)

Step 1 — Data + State

- Implement Interview + Message tables
- Implement state transitions

Step 2 — Document Handling

- Upload files
- Extract text (even naive extraction for training)

Step 3 — First LLM Integration

- Implement document analysis call
- Store match analysis

Step 4 — Interview Loop

- Ask question
- Store messages
- Score answer
- Generate next question

Step 5 — Report

- Aggregate scores
- Generate summary + recommendation

Step 6 (Stretch) — WebSockets and Better UX

- Real-time delivery
 - Better session management
-

Appendix A — What the Developer Should Demonstrate

- A working end-to-end flow from upload → analysis → interview → report
- Reliable LLM outputs via structured JSON
- Deterministic interview logic (state machine + difficulty adjustments)
- Persisted transcript with per-question scoring
- Basic integrity metadata captured and surfaced